Modelagem de Equalizadores Adaptativos para Comunicações Ópticas Coerentes com Processamento Acelerado por GPU

Marcos Eduardo Araújo [1], **Edson Porto da Silva** [2]

RESUMO

A otimização do processamento digital de sinais é essencial em sistemas de comunicações ópticas coerentes, que devem lidar com grandes volumes de dados em altas taxas e em tempo real. O desenvolvimento de circuitos integrados específicos (ASICs), principais responsáveis pelo processamento digital em receptores coerentes, exige simulações extensas e de alto custo computacional. Nesse contexto, o uso de GPUs tem se mostrado uma alternativa eficaz, permitindo acelerar simulações de modelos físicos complexos e reduzir significativamente o tempo de processamento. Este trabalho tem como objetivo o estudo e a implementação de modelos numéricos de equalização adaptativa aplicados a sistemas ópticos coerentes, explorando o potencial do processamento paralelo em GPU com a biblioteca CuPy. Foi realizada a implementação e avaliação de algoritmos como CMA, RDE e DD-LMS em Python. A etapa final envolve a comparação entre versões em CPU e GPU, visando quantificar os ganhos de desempenho obtidos. Os principais resultados obtidos demonstram uma melhoria significativa no desempenho computacional em ambas as versões (CPU e GPU), decorrente da otimização por vetorização em blocos, que reduziu o tempo de execução das operações matriciais. Além disso, verificou-se que o uso da GPU se torna particularmente vantajoso à medida que a complexidade do sinal aumenta, evidenciando seu potencial para aplicações em sistemas coerentes de alta taxa e larga escala.

Palavras-chave: comunicações ópticas, processamento digital de sinais, multiplexação espacial

[1] Aluno do curso de Engenharia Elétrica, Departamento de Engenharia Elétrica, UFCG, Campina Grande, PB, e-mail: marcos.araujo@ee.ufcg.edu.br

[2] Ph.D, Professor Adjunto, Departamento de Engenharia Elétrica, UFCG, Campina Grande, PB, e-mail: edson.silva@dee.ufcg.edu.br

Modelagem de Equalizadores Adaptativos para Comunicações Ópticas Coerentes com Processamento Acelerado por GPU

ABSTRACT

The optimization of digital signal processing is essential in coherent optical communication systems, which must handle large volumes of data at high rates and in real time. The development of application-specific integrated circuits (ASICs), which are primarily responsible for digital processing in coherent receivers, requires extensive and computationally expensive simulations. In this context, the use of GPUs has proven to be an effective alternative, enabling the acceleration of complex physical models and significantly reducing processing time. This work aims to study and implement numerical models of adaptive equalization applied to coherent optical systems, exploring the potential of parallel processing on GPUs using the CuPy library. Algorithms such as CMA, RDE, and DD-LMS were implemented and evaluated in Python. The final stage involves comparing CPU and GPU versions to quantify the performance gains achieved. The results demonstrate a significant improvement in computational performance in both implementations (CPU and GPU), mainly due to block vectorization optimization, which reduced matrix operation times. Furthermore, it was observed that the GPU becomes increasingly advantageous as signal complexity grows, highlighting its potential for high-rate and large-scale coherent communication systems.

Keywords: coherent optical communications, digital signal processing, spatial multiplexing

INTRODUÇÃO

A tecnologia de comunicações ópticas coerentes tornou-se fundamental para suportar a crescente demanda por alta velocidade e largura de banda nas redes de telecomunicações modernas. Contudo, a transmissão em enlaces de fibra de longa distância introduz distorções significativas no sinal, principalmente a Dispersão Cromática (CD, do inglês *Chromatic Dispersion*) e a Dispersão de Modo de Polarização (PMD, do inglês *Polarization Mode Dispersion*). Para superar esses desafios e garantir a recuperação da informação, são empregados equalizadores digitais, que utilizam algoritmos de Processamento de Sinais (DSP, do inglês *Digital Signal Processing*)(SAVORY, 2010) para mitigar ativamente os efeitos do canal.

Em sistemas de comunicações ópticas, a equalização é fundamental para mitigar distorções do canal, como a Interferência Intersimbólica (ISI, do inglês *Intersymbolic Interference*). Equalizadores estáticos aplicam uma correção predefinida, ideal para distorções conhecidas e estáveis. Em contraste, equalizadores adaptativos são sistemas dinâmicos que ajustam seus coeficientes de filtro iterativamente(DINIZ, 2012), aprendendo com o sinal recebido. Essa capacidade de adaptação os torna indispensáveis para compensar efeitos variáveis e de difícil modelagem, como a PMD, embora sua implementação exija um alto custo computacional em sistemas de alta velocidade.

Uma abordagem promissora para lidar com a complexidade computacional em sistemas de comunicações ópticas é a aceleração por meio de Unidades de Processamento Gráfico (GPUs, do inglês *Graphics Processing Units*), que são especializadas em processamento paralelo. Este projeto de iniciação tecnológica explora essa sinergia, propondo a implementação de equalizadores adaptativos na linguagem Python, com a aceleração via GPU proporcionada pela biblioteca CuPy. O objetivo é unir a simplicidade e a prototipagem rápida do Python com a capacidade de processamento massivo do CuPy, visando obter melhorias significativas de desempenho na simulação de algoritmos de equalização.

A proposta deste projeto inclui a investigação de diversos algoritmos de equalização adaptativa (FARUK; SAVORY, 2017), como o algoritmo do módulo constante (CMA, do inglês *Constant Modulus Algorithm*),o algoritmo do equalizador direcionado ao raio (RDE, do inglês *Radius-Directed Equalizer*), o algoritmo de mínimos quadrados médios orientado à decisão (DD-LMS, do inglês *Decision-Directed Least Mean Square*), focando em sua implementação otimizada em Python para processamento em GPUs.

Ao final deste projeto, espera-se validar a viabilidade técnica e quantificar os ganhos de desempenho da aceleração por GPU. Adicionalmente, o trabalho resultará em uma plataforma de simulação que poderá servir como base para estudos futuros e para a prototipagem de novos algoritmos de DSP para transceptores ópticos coerentes.

METODOLOGIA

Este estudo é motivado pelo desafio de desenvolver e implementar equalizadores adaptativos para serem usados em comunicações coerentes com aceleração de processamento via GPU, para encontar uma solução adequada precisamos primeiro investigar e selecionar algoritmos de equalizações adaptativas que possam ser usadas de forma eficiente para compensar distorções causadas pelo do canal de comunicações ópticas. Foi realizada uma revisão bibliográfica para selecionar os algoritmos de equalização adaptativa mais relevantes para sistemas ópticos coerentes, com foco nos algoritmos CMA, RDE e DD-LMS (SOARES, 2024).

Para realizarmos a configuração do ambiente de simulações, foi utilizado o software OptiCommPy (HERBSTER; SILVA, 2023) sendo open-sorce, o que nos possibilitou fazer algumas simulações de fibra óptica, permitindo a criação de cenários customizáveis e realistas. A metodologia partiu da implementação de um sistema de comunicação óptica coerente completo, utilizando o OptiCommPy, utilizando modulações de alta ordem (QAM, do inglês *Quadrature Amplitude Modulation*) com dupla polarização, propagação em fibra com dispersão e uma cadeia de DSP completa. Neste cenário mais complexo, o diagrama de constelação tornou-se a principal métrica visual para avaliar o impacto das distorções e, posteriormente, a eficácia dos equalizadores adaptativos em restaurar o sinal. Foi utilizada a detecção coerente que é definida pelo uso de um *Oscilador Local* (LO) no receptor para ser misturado com o sinal recebido, o que permite extrair a informação de amplitude e fase como ilustrado na Figura 1.

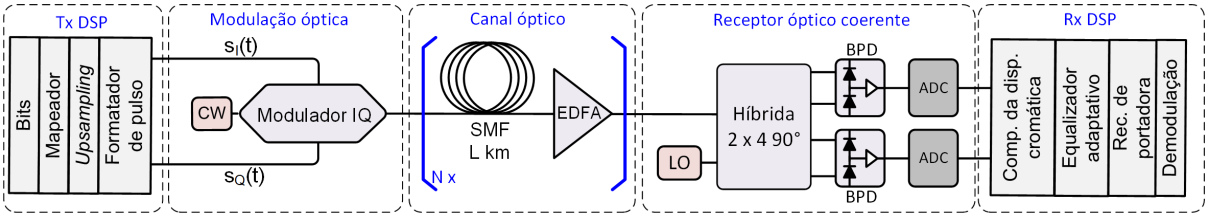


Figura 1 – Esquema simplificado de um sistema de comunicação óptica coerente. Fonte: (SILVA, 2007)

A fase inicial do projeto foi dedicada à construção e validação de um sistema de comunicação óptica coerente, utilizando a biblioteca OptiCommPy como base. A escolha dos parâmetros foi fundamental para estabelecer um cenário de teste realista e de alta capacidade, incluindo uma modulação 64-QAM com dupla polarização, operando a uma alta taxa de símbolos de 32 Gbaud. Para maximizar a eficiência espectral, foi implementado um pulso (RRC, do inglês *Root-Raised Cosine*) com um fator de *roll-off* agressivo de 0.01, uma técnica comum em sistemas modernos. Foi utilizada uma função para gerar o sinal de transmissão ideal, que serviu como referência para todas as análises de desempenho subsequentes.

No lado do receptor, a simulação foi projetada para emular fielmente a cadeia de

processamento. Utilizou-se uma função para modelar não apenas a detecção coerente, mas também o principal efeito da PMD através de uma rotação de polarização. O sinal recebido passou então por um filtro casado, essencial para maximizar a SNR, e pela *Compensação Eletrônica de Dispersão* (EDC), o primeiro estágio de equalização para corrigir o alargamento temporal dos pulsos. Finalmente, funções como decimate foram usadas para ajustar a taxa de amostragem do sinal, preparando-o para a etapa de equalização adaptativa. Com o sinal recebido já contendo os efeitos combinados de Dispersão Cromática e de Modo de Polarização, a tarefa de recuperação de dados foi delegada ao equalizador. Para este fim, foi adotada a arquitetura MIMO 2x2, uma escolha imprescindível para sistemas com dupla polarização. Esta estrutura, conhecida como "*butterfly equalizer*"e ilustrada na Figura 2, é projetada especificamente para "desembaralhar"os sinais das polarizações X e Y, corrigindo o crosstalk e a distorção intersimbólica.

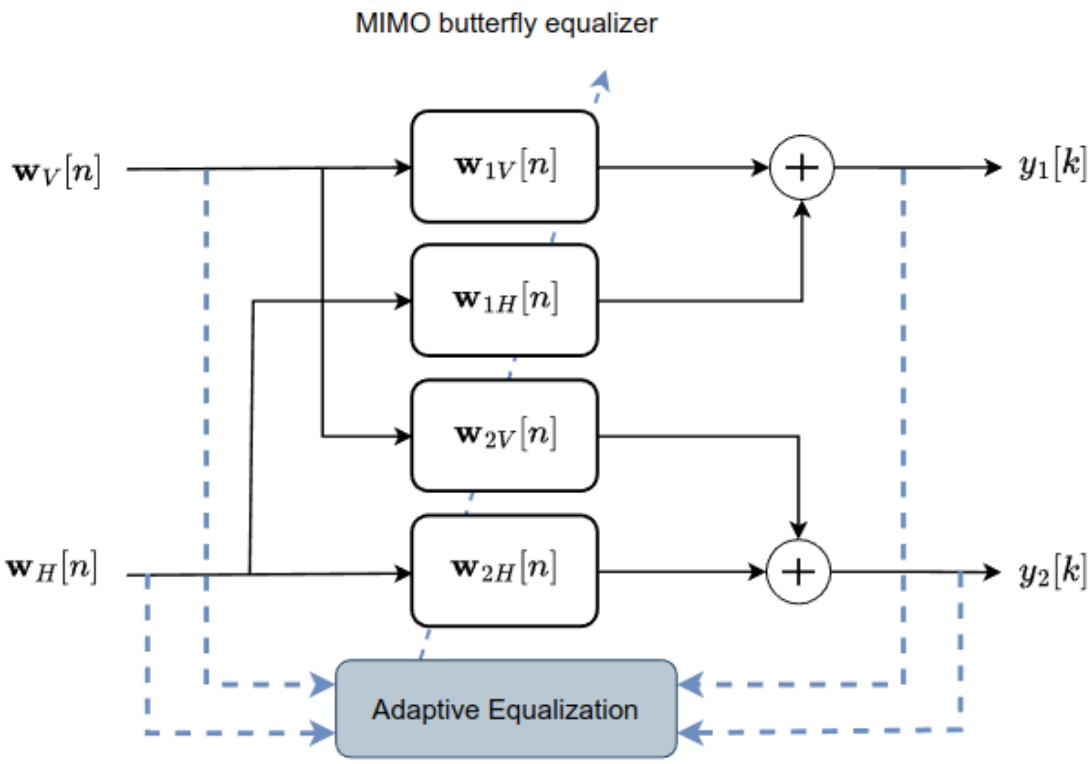


Figura 2 – Arquitetura "butterfly"do equalizador adaptativo MIMO 2x2. Fonte: (SOARES, 2024)

Validada a implementação base em NumPy, a otimização dos equalizadores adaptativos foi realizada em duas etapas. Primeiramente, o código foi portado para CuPy, habilitando a execução na arquitetura CUDA. O passo seguinte e mais significativo foi a refatoração dos algoritmos de uma lógica amostra-por-amostra para um modelo de processamento em blocos, implementado tanto em NumPy quanto em CuPy para uma comparação justa. Esta re-arquitetura envolveu técnicas avançadas

para pré-calcular as matrizes de entrada do filtro, transformando a computação em operações de matrizes paralelizáveis.

A validação final quantificou os ganhos desta abordagem. Em seguida, um benchmark de performance foi conduzido, aumentando a carga de trabalho para medir a escalabilidade. A análise dos gráficos comparativos de tempo de execução demonstrou a significativa aceleração proporcionada pela GPU, validando a eficácia da metodologia de otimização e atingindo os objetivos propostos de aceleração de hardware para a simulação de sistemas ópticos coerentes.

DESENVOLVIMENTO

O desenvolvimento da plataforma de simulação foi estruturado de forma modular, seguindo o diagrama de blocos de um sistema de comunicação óptica coerente, conforme ilustrado na Figura 1. Cada subsistema — transmissor, canal e receptor — foi implementado em **Python**, utilizando a biblioteca OptiCommPy como base para as funcionalidades de processamento de sinal.

Transmissor e Geração de Sinal (Tx)

A primeira etapa consiste na geração do sinal digital a ser transmitido. Para isso, foi utilizada uma função configurada para criar um cenário de teste realista e de alta capacidade. Os principais parâmetros de transmissão foram definidos da seguinte forma:

- Modulação: Foi escolhida a modulação por amplitude em quadratura de 64 símbolos (64-QAM), com dupla polarização, visando uma alta taxa de bits.
- Taxa de Símbolos: A simulação operou a uma alta taxa de 32 Gbaud.
- Formatação de Pulso: Para maximizar a eficiência espectral, foi implementado um pulso RRC com um fator de *roll-off* agressivo de 0.01, uma técnica comum em sistemas modernos.
- Duração da Simulação: O número de bits foi calculado para garantir um total de 200.000 símbolos por polarização, assegurando relevância estatística para as análises de desempenho.

A saída desta etapa é uma forma de onda elétrica, que seria então convertida em um sinal óptico por um modulador IQ, (do inglês *In-Phase and Quadrature*), conforme o diagrama de blocos.

Simulação do Canal Óptico

Para emular a propagação do sinal em um enlace de longa distância, foi implementado um modelo de canal óptico que inclui os principais efeitos de degradação:

- Fibra Óptica: A propagação pela fibra foi modelada pela função linearFiberChannel, configurada para um enlace de 300 km com um parâmetro de dispersão cromática de 17 ps/nm/km. Este efeito é o principal responsável pela ISI em sistemas de alta velocidade.

- Amplificação e Ruído: Para simular um sistema realista onde as perdas de potência são compensadas, foram utilizados modelos de amplificação e ruído. A função responsável pela amplificação simula um amplificador óptico que adiciona ganho e ruído ASE(do inglês *Amplified Spontaneous Emission noise*), enquanto a função que simula o ruído permite a adição de ruído gaussiano para simular uma Relação Sinal-Ruído (SNR) específica no sistema.

Este ambiente de simulação completo permitiu a geração de um sinal recebido realista, com todas as distorções que os equalizadores adaptativos visam corrigir.

Recepção Coerente e Degradação do Sinal

Após a propagação pelo canal de fibra, o sinal óptico, já degradado pela CD, chega ao receptor. Nesta etapa, são modelados os componentes do *front-end* de um receptor óptico coerente e as imperfeições adicionais que ele introduz, preparando o sinal para o processamento digital (DSP).

Primeiramente, é gerado o sinal do Oscilador Local (LO), que serve como referência para a demodulação. Na simulação, o LO foi modelado para incluir imperfeições realistas que afetam o desempenho do sistema:

- Ruído de Fase: Foi utilizada uma função configurada para simular um laser com uma largura de linha de 100 kHz, introduzindo flutuações de fase estocásticas.

- Deslocamento de Frequência: Foi adicionado um deslocamento de frequência (FO, do inglês *Frequency Offset*) de 150 MHz entre a portadora do sinal e o LO. Este desalinhamento, comum em sistemas reais, causa uma rotação contínua na constelação recebida.

Em seguida, uma função que emula a detecção coerente com diversidade de polarização. Além de simular a mistura do sinal com o LO na Híbrida Óptica e a conversão para o domínio elétrico nos fotodetectores, esta função foi utilizada para introduzir o principal efeito da Dispersão de Modo de Polarização (PMD). Ao configurar um ângulo de rotação de polarização $\theta_{sig} = \pi/3$, a função simula uma severa mistura entre os sinais das polarizações X e Y (*crosstalk*). Ao final desta etapa, o sinal elétrico contém a superposição de todas as distorções do enlace: CD, PMD, ruído de fase e deslocamento de frequência, representando fielmente o sinal desafiador que a cadeia de DSP subsequente deve corrigir.

Pré-processamento e Compensação de Dispersão no Rx DSP

A etapa inicial do Rx DSP é responsável pelo pré-processamento e pela compensação da distorção linear mais severa do canal: a Dispersão Cromática (CD).

A Dispersão Cromática (CD) é um fenômeno que ocorre porque os diferentes comprimentos de onda (ou "cores") que compõem um pulso óptico viajam em velocidades ligeiramente diferentes na fibra. Como resultado, componentes do mesmo pulso chegam ao receptor em momentos distintos, causando um alargamento temporal, como ilustrado na Figura 3. Este alargamento é a principal causa da Interferência Intersimbólica (ISI), onde um símbolo "borra"e interfere com seus vizinhos, degradando severamente o sinal.

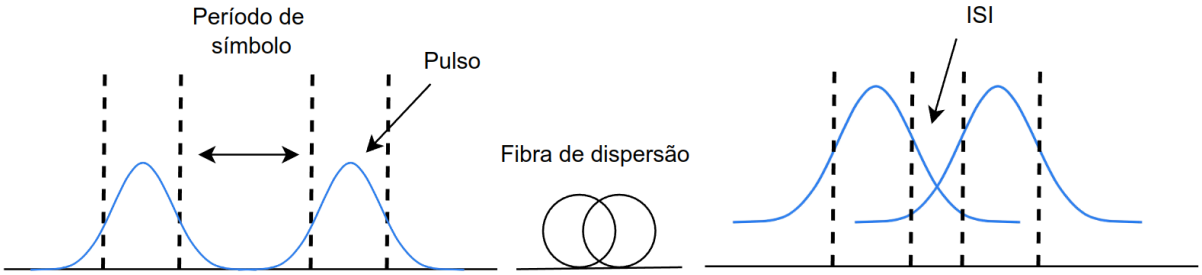


Figura 3 – Ilustração do alargamento de pulso e da Interferência Intersimbólica (ISI) causados pela Dispersão Cromática. Fonte: (SOARES, 2024)

A primeira e mais importante etapa é a filtragem casada, onde um filtro FIR, cuja resposta ao impulso é uma cópia idêntica do pulso RRC utilizado no transmissor, é aplicado ao sinal. Esta operação é crucial para duas finalidades: maximizar a Relação Sinal-Ruído antes das etapas subsequentes e completar a formatação de pulso para um sistema Cosseno Levantado (RC, do inglês *raised cosine*) de ponta a ponta, mitigando a Interferência Intersimbólica (ISI).

Logo após, o sinal é processado pelo bloco de Compensação Eletrônica de Dispersão (EDC, do inglês *Electronic Dispersion Compensation*). A função aplica um filtro digital estático cuja resposta em frequência é o inverso exato do efeito da dispersão acumulada na fibra. Utilizando os parâmetros do enlace, este bloco corrige o alargamento temporal dos pulsos, que é a distorção linear mais severa do canal. Finalmente, o sinal é preparado para os algoritmos adaptativos: a função decimate ajusta a taxa de amostragem para uma amostra por símbolo, e a pnorm normaliza a potência do sinal. Ao final deste processo, o sinal de saída está com a dispersão cromática compensada e pronto para a etapa de equalização adaptativa MIMO.

Arquitetura do Equalizador Adaptativo MIMO 2x2

Em sistemas coerentes modernos, dois sinais independentes são transmitidos em polarizações ortogonais (X e Y) para dobrar a taxa de dados, uma técnica

conhecida como PDM. No entanto, a PMD na fibra causa uma mistura entre esses sinais. Para compensar a mistura de polarizações (PMD), todos os algoritmos foram implementados sobre uma estrutura de equalizador MIMO (do inglês *Multiple-Input Multiple-Output*) 2x2, esta arquitetura conhecida como *butterfly* e ilustrada na Figura 2, utiliza quatro filtros FIR (do inglês *Finite Impulse Response*) adaptativos independentes (`w_xx, w_xy, w_yx, w_yy`) para processar as duas polarizações recebidas (entradas) e reconstruir os dois sinais originais (saídas), desfazendo o acoplamento entre os modos. A atualização dos coeficientes (w) de cada filtro é governada pela equação de gradiente descendente estocástico, onde a definição do erro e(k) varia conforme o algoritmo utilizado. Foram implementados três algoritmos de equalização adaptativa, cada um com uma estratégia diferente para o cálculo do erro, formando uma cadeia de processamento robusta.

O CMA é um algoritmo cego, utilizado para a convergência inicial do equalizador. Ele ajusta os coeficientes do filtro para minimizar o desvio da potência instantânea da saída $(|y(k)|^2)$ em relação a uma potência de referência constante (R), calculada estatisticamente a partir da constelação. Sua principal função nesta implementação é "abrir o olho"da constelação, separando as polarizações e compensando o deslocamento de frequência grosseiro, sendo ideal para formatos de modulação com amplitude constante, como PSK (QPSK, 8-PSK).

O RDE é uma evolução do CMA, otimizado para constelações de múltiplos anéis, como a QAM. Em vez de usar um raio de referência fixo, o RDE primeiro faz uma "pré-decisão"para identificar a qual dos raios ideais da constelação $(Rrde)$ o símbolo de saída $y(k)$ provavelmente pertence. Em seguida, ele utiliza o raio decidido para calcular o erro. Esta abordagem permite um ajuste de amplitude mais preciso, sendo utilizada como o segundo estágio na estratégia de pré-convergência cma-to-rde.

O DD-LMS é um algoritmo guiado por decisão, ativado após a convergência cega inicial. Ele opera fazendo uma "decisão brusca"para encontrar o símbolo da constelação ideal $(d(k))$ mais próximo da saída do equalizador $(y(k))$. O erro é então calculado como a diferença vetorial $e(k) = d(k) - y(k)$. Esta abordagem é utilizada para o rastreamento fino do sinal, minimizando o erro residual e rastreando variações lentas do canal, sendo crucial para alcançar um baixo BER(do inglês *Bit Error Rate*).

Para obter os resultados, foi utilizada uma função gerenciadora (mimoAdaptEq) que implementa a cadeia de DSP completa, conforme o código abaixo. A otimização para GPU foi realizada através da biblioteca CuPy, refatorando a lógica de processamento amostra-por-amostra para uma arquitetura em blocos, o que permitiu uma comparação de desempenho justa entre as plataformas de CPU e GPU.

```
def mimoAdaptEq(x, paramEq):
.
```

```
.
.
    if paramEq.alg == "cma":
        y, e, w = cmaUp(x, constSymb, nModes, paramEq)
    elif paramEq.alg == "rde":
        y, e, w = rdeUp(x, constSymb, nModes, paramEq)
    elif paramEq.alg == "cma-to-rde":
        y, e, w = cmaUp(x, constSymb, nModes, paramEq, preConv=
           True)
    elif paramEq.alg == "dd-lms":
        y, e, w = ddlmsUp(x, constSymb, nModes, paramEq)
    else:
        raise ValueError("Algoritmo eq. especificado incorreto.")

    return y, e, w
```

Após o pré-processamento e a compensação da dispersão estática, o sinal x está pronto para a etapa de equalização adaptativa, o coração do DSP deste projeto. Foi implementada uma estratégia de dois estágios para garantir uma convergência robusta. Primeiramente, um equalizador cego, o cma-to-rde, é aplicado para realizar a separação das polarizações e a convergência inicial, transformando a constelação disforme em anéis estáticos, porém ruidosos.

Em seguida, o sinal passa por um bloco de Recuperação de Fase da Portadora (CPR, do inglês *Carrier Phase Recovery*), utilizando o algoritmo (BPS, do inglês *Blind Phase Search*). Esta etapa é crucial para remover o ruído de fase e o deslocamento de frequência residual, fazendo com que os anéis "colapsem"para formar a grade da constelação QAM. Finalmente, um equalizador DD-LMS é utilizado para o rastreamento fino do sinal, minimizando o erro residual. A eficácia de cada estágio foi monitorada através da visualização da constelação e da análise da curva de erro (|e[n]|) em função das iterações, como mostrado nas figuras de resultados.

Sincronização e Análise de Desempenho

Para validar quantitativamente o desempenho de toda a cadeia de DSP, foi implementada uma rotina final de análise. O primeiro passo crítico nesta rotina é a sincronização temporal. Como os diversos filtros do sistema introduzem um atraso significativo, uma função foi utilizada para alinhar perfeitamente a sequência de símbolos recebida e processada (rx) com a sequência de referência transmitida (tx), eliminando o atraso para uma comparação justa.

Com os sinais devidamente sincronizados, uma função foi empregada para calcular as três métricas de desempenho fundamentais:

- Signal to Noise Ratio (SNR): A Relação Sinal-Ruído, estimada a partir da magnitude do vetor de erro da constelação final. Ela quantifica a "limpeza"do sinal recebido.
- Symbol Error Rate (SER): A Taxa de Erro de Símbolo, que mede a frequência com que um símbolo QAM inteiro é decidido erroneamente.
- Bit Error Rate (BER): A Taxa de Erro de Bit, a métrica de desempenho final mais importante. Ela compara a sequência de bits recuperada com a sequência original para calcular a taxa de erros de bit.

Otimização dos Algoritmos por Vetorização em Blocos

Após a validação funcional dos algoritmos de equalização em uma implementação amostra-por-amostra, o foco do projeto se voltou para a otimização de performance. A abordagem sequencial, embora didática, possui um gargalo de desempenho significativo devido à sobrecarga de execução de um loop for em Python por centenas de milhares de iterações. Para superar essa limitação, foi implementada uma refatoração para vetorização em blocos, tanto para a versão em CPU (NumPy) quanto para a GPU (CuPy).

A principal mudança arquitetônica foi a eliminação do loop amostra-por-amostra em favor de um loop que itera sobre grandes blocos de dados. Dentro de cada iteração, as operações de filtragem e atualização de pesos foram transformadas em operações de matrizes de alta performance. A etapa chave para viabilizar essa vetorização foi a criação de uma função auxiliar, que utiliza a técnica de `stride_tricks` `(numpy/cupy).lib.stride_tricks.as_strided` para pré-calcular todas as janelas de entrada do filtro para um bloco de uma só vez, de forma extremamente eficiente e sem a necessidade de cópias de memória.

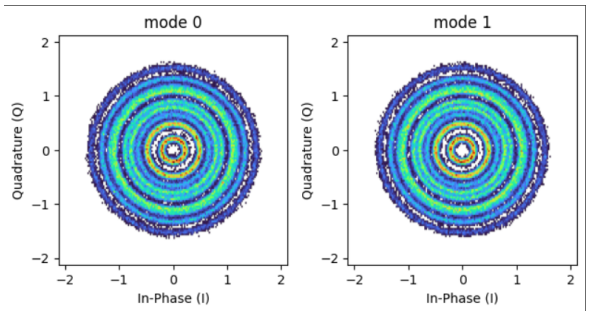
Com as matrizes de janela de entrada `(xV e xH)` pré-calculadas, a operação de filtragem para um bloco inteiro foi reduzida a multiplicações de matrizes `(@ ou .dot)`, e a atualização dos pesos foi calculada com base no gradiente médio do bloco. Esta abordagem delega os cálculos massivos para as rotinas de álgebra linear otimizadas do NumPy e do CuPy (em CUDA), minimizando drasticamente o controle sequencial da CPU. O ganho de performance com esta mudança foi gigantesco: simulações que antes duravam minutos passaram a ser executadas em menos de um segundo, tanto em NumPy quanto em CuPy.

RESULTADOS E DISCUSSÕES

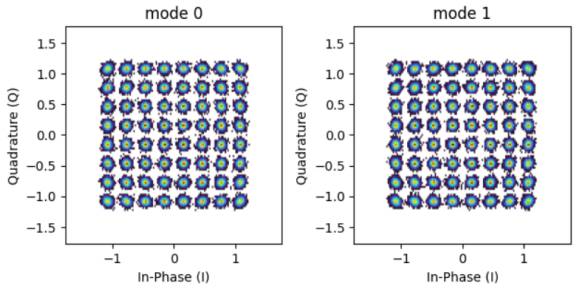
Nesta seção, são apresentados os resultados da simulação do sistema de comunicação óptica coerente descrito na metodologia. Para ilustrar a eficácia da cadeia de DSP, foi selecionado um estudo de caso com a transmissão de 2.10^5 símbolos. A

análise subsequente detalha a evolução da qualidade do sinal após cada bloco de processamento principal.

A avaliação visual da constelação em diferentes pontos do receptor é uma ferramenta poderosa para diagnosticar o impacto de cada distorção e a eficácia de cada algoritmo de correção. As figuras abaixo ilustram a transformação do sinal 64-QAM ao longo da cadeia de DSP.



(a) Saída do equalizador cma-to-rde, com a estrutura de anéis restaurada.



(b) Saída final após o CPR, com a grade 64-QAM recuperada.

Figura 4 – Resultados do processo de equalização CMA/RDE.

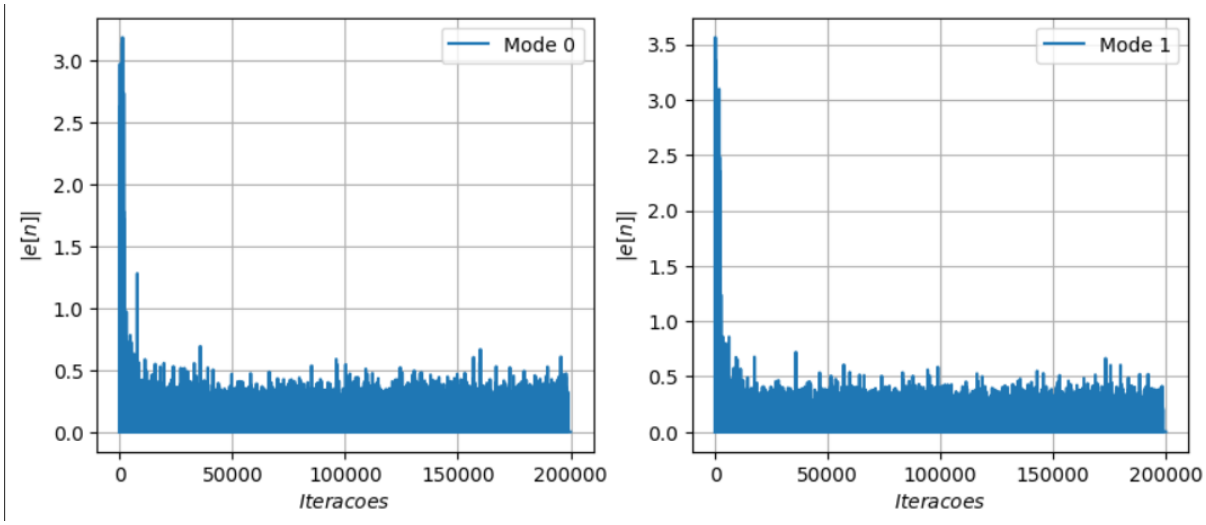


Figura 5 – Evolução do erro do CMA/RDE por iteração para os modos de polarização X e Y durante a equalização.

Após a aplicação do equalizador cego cma-to-rde, a estrutura de amplitude do sinal é recuperada, resultando na formação de anéis estáticos, porém ainda borrados pelo ruído de fase, como mostra a Figura 4a. A etapa final de Recuperação de Fase da Portadora (CPR) é, portanto, essencial. Como visto na Figura 4b, o algoritmo BPS remove o ruído de fase residual, fazendo com que os anéis "colapsem"nos 64 pontos da constelação, resultando em um sinal "limpo"e pronto para a demodulação.

A convergência do equalizador adaptativo pode ser observada através da curva de erro, mostrada na Figura 5. O gráfico de $e[n]$ versus o número de iterações demonstra

uma queda abrupta inicial, seguida pela estabilização em um piso de ruído baixo, caracterizando um processo de adaptação bem-sucedido para ambos os modos de polarização.

Com a fase da portadora já recuperada, o sinal é então processado pelo último estágio de equalização: um filtro adaptativo operando no modo Decision-Directed Least Mean Square (DD-LMS). A principal função desta etapa é realizar o ajuste fino da constelação 6. Como o sinal de entrada já possui um erro baixo o suficiente para que as decisões sobre os símbolos sejam confiáveis, o DD-LMS utiliza essas decisões como referência para minimizar o erro quadrático médio residual 7. Isso resulta em um refinamento da posição dos símbolos, deixando o sinal pronto para as etapas finais de sincronização e demodulação.

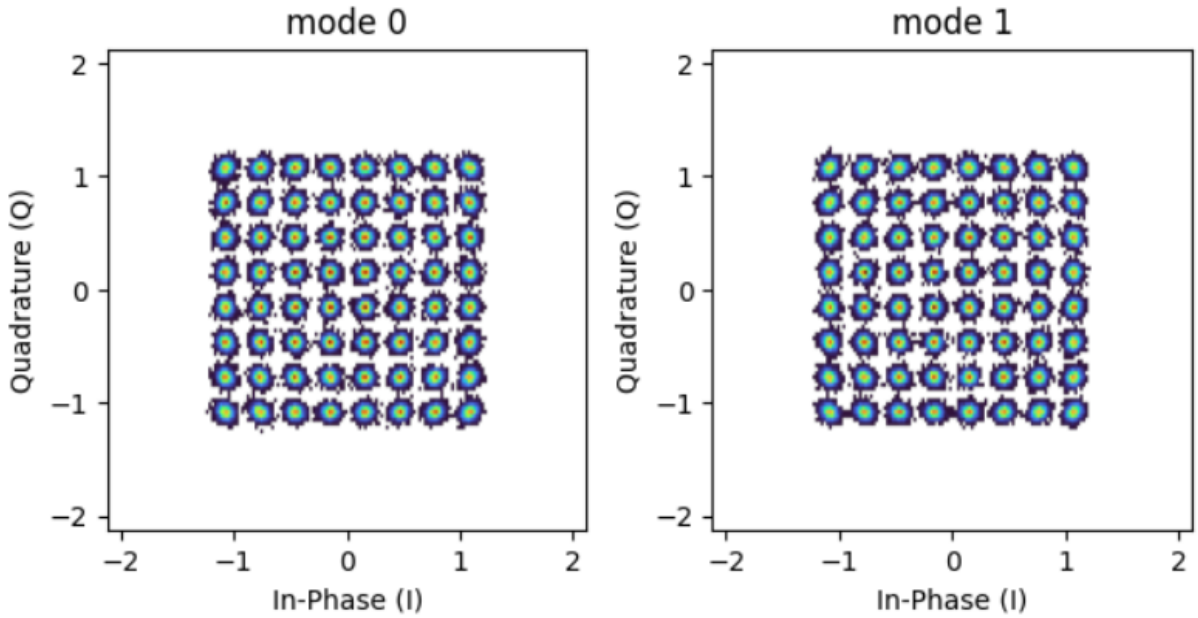


Figura 6 – Saída do equalizador DD-LMS

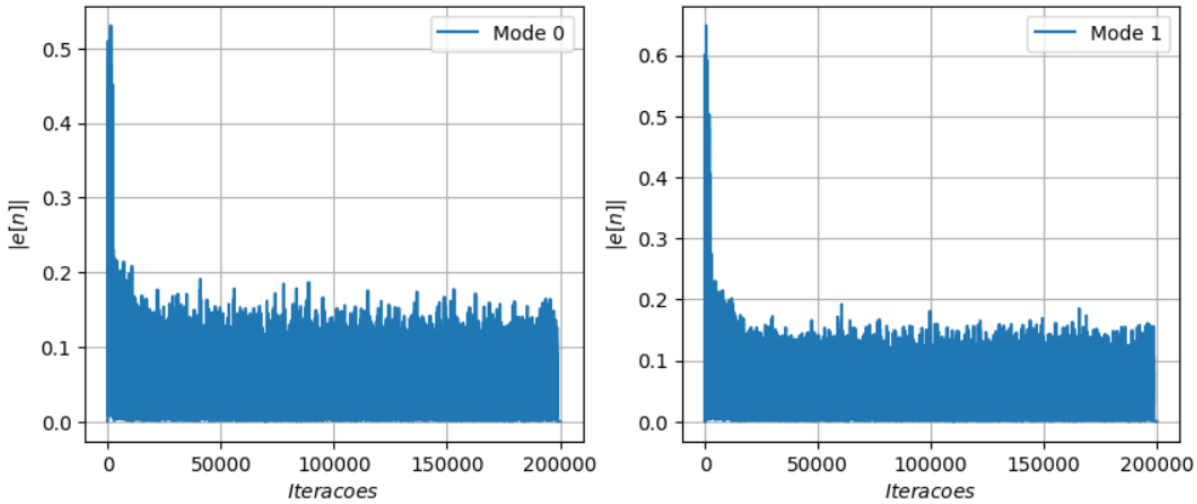


Figura 7 – Evolução do erro do DD-LMS por iteração para os modos de polarização X e Y durante a equalização.

A eficácia de toda a cadeia de processamento de sinais — desde a compensação de dispersão até a equalização adaptativa e a recuperação de fase — é finalmente validada pela comparação direta entre o sinal transmitido (Tx) e o sinal recuperado (Rx) no domínio do tempo. A 8 ilustra essa comparação para os primeiros 100 símbolos de cada polarização.

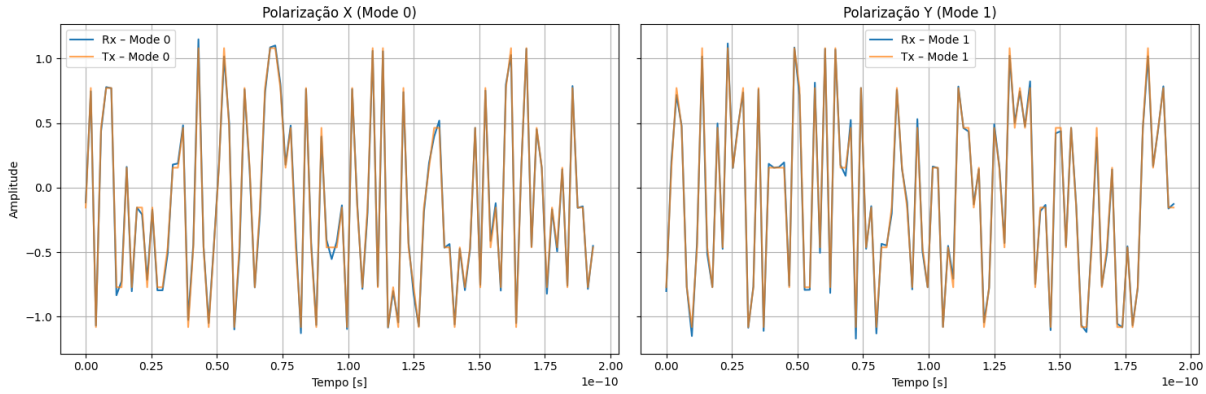


Figura 8 – Comparação entre o sinal transmitido ideal (Tx - Laranja) e o sinal recuperado final (Rx - Azul) para as polarizações X (esquerda) e Y (direita).

Como pode ser observado, o sinal Rx, após passar por todas as etapas de correção, segue a forma de onda do sinal Tx original com altíssima fidelidade. As pequenas diferenças remanescentes entre as duas curvas representam o erro residual do sistema, composto por ruído e distorções não completamente compensadas.

Esta forte correlação visual entre o sinal enviado e o recuperado é a manifestação direta das excelentes métricas de desempenho quantitativo alcançadas, como a alta SNR e a baixa BER. Isso demonstra que o conjunto de algoritmos de DSP implementado foi capaz de reverter com sucesso as severas degradações impostas

pelo canal de fibra óptica, reconstruindo a informação com a precisão necessária para uma comunicação confiável.

Benchmark de Desempenho em Ambiente de Nuvem (Google Colab)

Para quantificar a eficácia da aceleração por hardware, foi realizado um benchmark de desempenho em um ambiente de nuvem utilizando o Google Colab. As especificações do ambiente de teste foram: CPU Intel Xeon @ 2.2GHz, 12 GB de RAM e uma GPU NVIDIA Tesla T4 com 15 GB de VRAM (CUDA 12.4). Foram comparadas as implementações otimizadas em blocos para CPU (NumPy) e GPU (CuPy) dos equalizadores CMA/RDE e DD-LMS.

O equalizador cego é o bloco computacionalmente mais intensivo da cadeia de DSP. 9 apresenta o tempo de execução para as versões CPU e GPU em função do número de símbolos simulados.

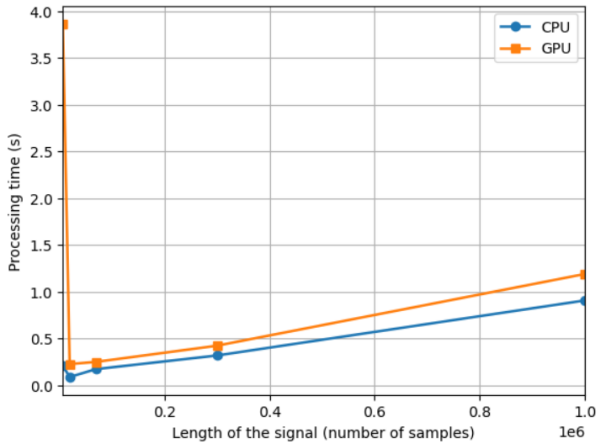


Figura 9 – Comparativo de Desempenho do Equalizador CMA/RDE (CPU vs. GPU)

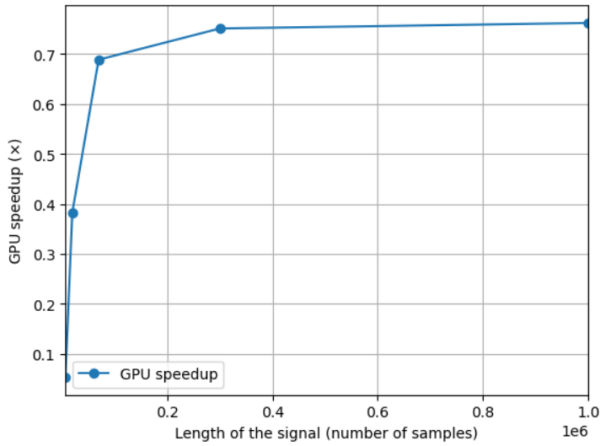


Figura 10 – Ganho de Velocidade (Speedup) da GPU para o Equalizador CMA/RDE

A análise do gráfico revela que para sinais muito curtos, o tempo de execução de ambas as plataformas é similar, com a CPU apresentando uma ligeira vantagem

devido à sobrecarga de comunicação com a GPU. No entanto, à medida que a carga de trabalho aumenta, a superioridade da arquitetura paralela da GPU se torna evidente. Para simulações com mais de 10^5 amostras, a implementação em CuPy se mostra significativamente mais rápida. O ganho de velocidade (*speedup*), calculado como a razão entre o tempo da CPU e o tempo da GPU, é apresentado na 10

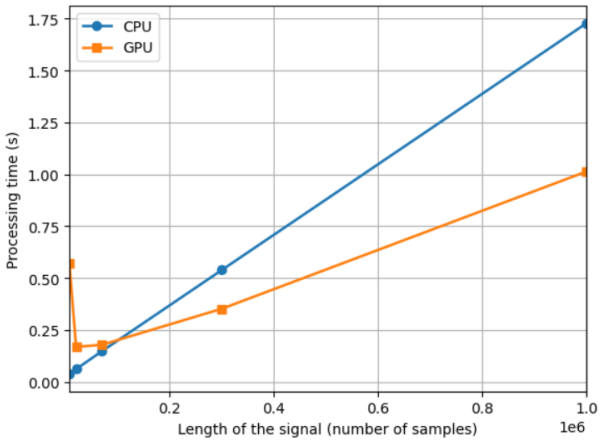


Figura 11 – Comparativo de Desempenho do Equalizador DD-LMS (CPU vs. GPU)

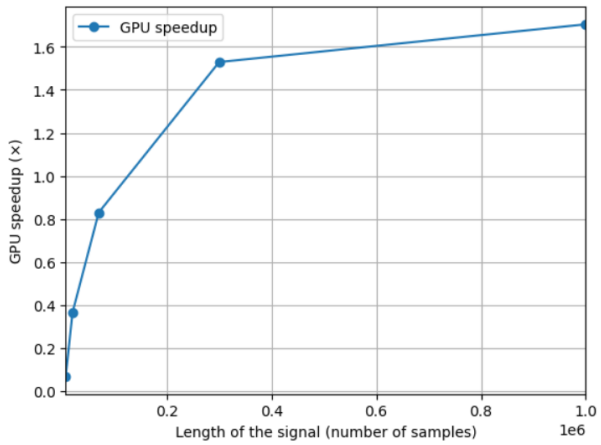


Figura 12 – Ganho de Velocidade (Speedup) da GPU para o Equalizador DD-LMS

Observa-se que o speedup cresce com o tamanho do problema, atingindo um ganho de aproximadamente 0.8x em cma-to-rde e 1.7x DD-LMS para simulações com um milhão de amostras, confirmando a excelente escalabilidade da abordagem em GPU. Uma análise similar foi conduzida para o estágio de rastreamento fino com o algoritmo DD-LMS11. Embora computacionalmente mais simples que o cma-to-rde, a aceleração por GPU ainda se mostra vantajosa para grandes volumes de dados.

A análise do gráfico na 12 demonstra claramente a vantagem da aceleração por GPU. Conforme a carga de trabalho aumenta, a implementação em CuPy escala de forma muito mais eficiente que a versão em NumPy. Isso valida a eficácia da arquitetura

de processamento em blocos na GPU para reduzir drasticamente o tempo de simulação de equalizadores adaptativos.

Benchmark de Desempenho em Ambiente Local (PC)

Para complementar a análise, um segundo benchmark foi conduzido em um ambiente de hardware local de alta performance. As especificações do sistema de teste foram: CPU Intel Xeon(R) W9-3495X, 1 TB de RAM e uma GPU NVIDIA RTX 4500 com 24 GB de VRAM (CUDA 12.8).

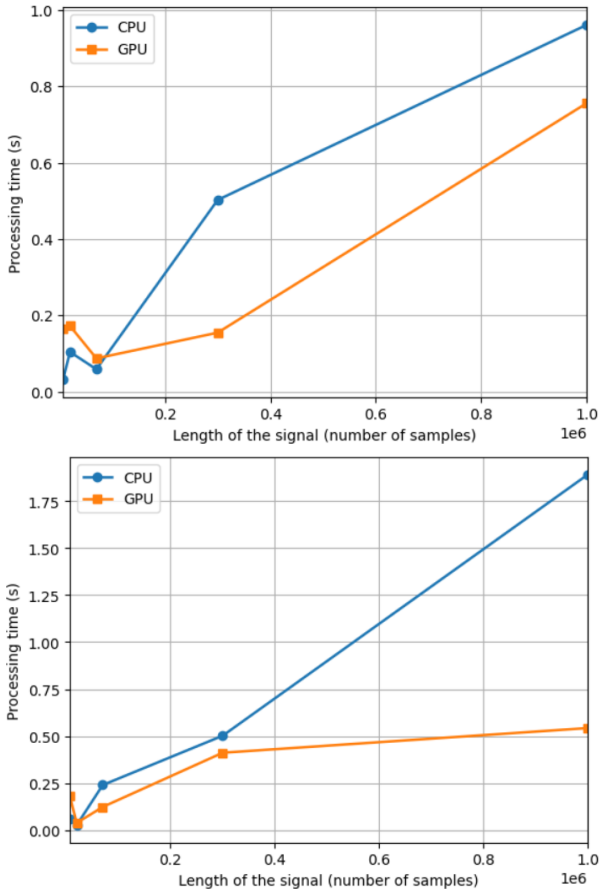


Figura 13 – Comparativo de Desempenho (CPU vs. GPU) no PC Local

A 13 apresenta o comparativo de tempo de execução entre as implementações em CPU (NumPy) e GPU (CuPy) no hardware local. Os resultados demonstram que a CPU Xeon, por ser um processador de alta performance, apresenta um tempo de execução muito competitivo para cargas de trabalho menores. No entanto, a tendência observada no ambiente de nuvem se repete: à medida que o número de símbolos da simulação aumenta, a implementação em GPU escala de forma mais eficiente, e a curva de tempo de execução se torna significativamente mais plana em comparação com a da CPU.

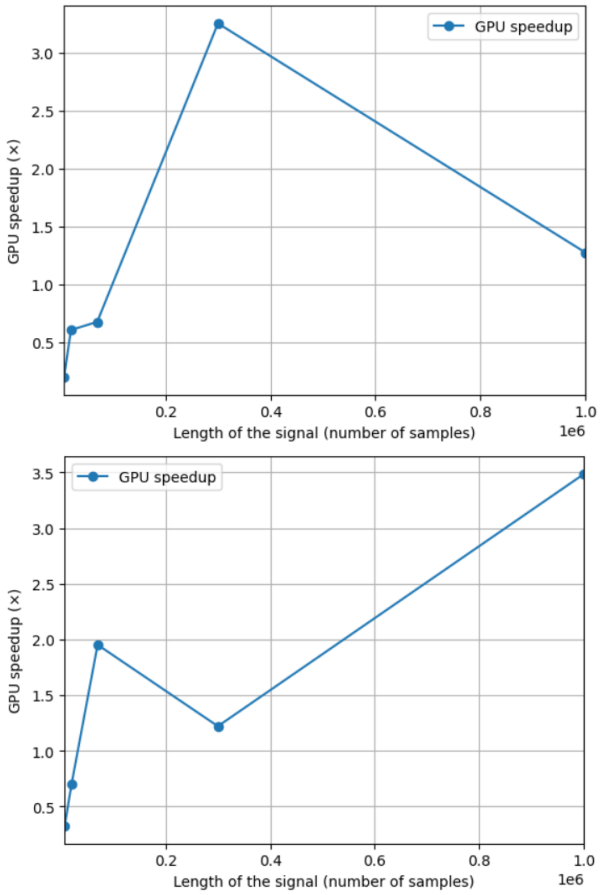


Figura 14 – Ganho de Velocidade (*Speedup*) da GPU para o Equalizador no PC Local

O ganho de velocidade (*speedup*) da GPU em relação à CPU é ilustrado na 14. O gráfico confirma que, para problemas de maior dimensão (acima de 3.10^5 amostras), o speedup se torna substancial no equaliador DD-LMS. Isso ocorre porque o tempo economizado pelo processamento massivamente paralelo da GPU supera a sobrecarga de comunicação entre a CPU e a GPU, que é mais impactante em simulações curtas. Os resultados validam que a arquitetura de processamento em blocos implementada em CuPy é uma estratégia eficaz para a aceleração de equalizadores adaptativos, proporcionando uma redução significativa no tempo de simulação, o que é crucial para o ciclo de desenvolvimento e otimização de sistemas ópticos coerentes.

CONCLUSÃO

Neste projeto, comparou-se o desempenho de implementações de equalizadores adaptativos em CPU (NumPy) e GPU (CuPy), especificamente para o problema de mitigação de distorções em sistemas de comunicações ópticas coerentes. Os resultados desta pesquisa destacaram a enorme eficácia da aceleração por GPU para simulações computacionalmente intensivas. Além disso, foi observado que a arquitetura do algoritmo, seja ela amostra-por-amostra ou em blocos, tem um impacto fundamental tanto na performance computacional quanto na estabilidade da convergência.

Foi possível concluir que, para obter ganhos de performance significativos, a migração de uma lógica sequencial para uma arquitetura de processamento em blocos vetorizado é o passo crucial. Embora esta abordagem exija um ajuste mais cuidadoso de parâmetros como o passo de adaptação e o comprimento do filtro para garantir a estabilidade, ela se mostrou a única forma viável de extrair o potencial do processamento paralelo. Esta otimização resultou em ganhos de velocidade expressivos não apenas na GPU, mas também na implementação em CPU.

Observando o comportamento das plataformas quando submetidas a cargas de trabalho crescentes, vemos que a implementação em CPU com NumPy é adequada para a validação funcional e para simulações de pequena escala. Contudo, a análise de escalabilidade demonstrou que seu tempo de execução cresce de forma acentuada com o número de símbolos. Em contraste, a versão em GPU com CuPy, embora possua uma sobrecarga inicial, apresentou uma escalabilidade muito superior, tornando-se ordens de magnitude mais rápida para os grandes volumes de dados que uma análise estatística robusta (como o cálculo de BER) exige.

Portanto, para o ciclo de desenvolvimento de novas gerações de transceptores ópticos, a utilização de plataformas de simulação aceleradas por GPU em linguagens de alto nível como Python se mostra uma estratégia promissora. Ela permite a prototipagem e a otimização rápida de algoritmos de DSP complexos, reduzindo significativamente o tempo entre o conceito teórico e a validação. A capacidade de testar exaustivamente diferentes configurações de equalizadores em minutos, em vez de horas ou dias, acelera a inovação e pode levar ao desenvolvimento de ASICs mais eficientes e robustos para futuras redes de comunicação.

AGRADECIMENTOS

O presente trabalho foi realizado com apoio do CNPq, Conselho Nacional de Desenvolvimento Científico e Tecnológico – Brasil". Sendo desenvolvido através do programa de PIBITI/CNPq-UFCG.

REFERÊNCIAS

DINIZ, P. S. R. **Adaptive Filtering: Algorithms and Practical Implementation**. 4. ed. Springer US, 2012. ISBN 978-1-4614-4105-2. Disponível em: <https://link.springer.com/book/10.1007/978-1-4614-4106-9>. páginas 3

FARUK, M. S.; SAVORY, S. J. Digital signal processing for coherent transceivers employing multilevel formats. **Journal of Lightwave Technology**, v. 35, n. 5, p. 1125–1141, mar. 2017. ISSN 1558-2213. Disponível em: <https://ieeexplore.ieee.org/document/7812771>. páginas 3

HERBSTER, J. M. C. D. F. d. S. S. A.; SILVA, E. P. da. **OptiCommPy: Fiber Optic Communications with Python**. 2023. <https://github.com/edsonportosilva/OptiCommPy>. GitHub repository. páginas 4

SAVORY, S. J. Digital coherent optical receivers: Algorithms and subsystems. **IEEE Journal of Selected Topics in Quantum Electronics**, v. 16, n. 5, p. 1164–1179, 2010. ISSN 1558-4542. páginas 3

SILVA, E. P. da. **OpticalCommunications**. 2007. <https://github.com/edsonportosilva/OpticalCommunications/tree/main>. GitHub repository. páginas 4

SOARES, E. P. d. S. S. J. B. **CoherentOptics**. 2024. <https://github.com/silasabs/CoherentOptics/tree/main/examples>. GitHub repository. páginas 4, 5, 8